

Precision-First Entity Resolution

A differentiated phase plan for the Amazon ML Challenge 2026

The strategic idea

Most entity-resolution pipelines treat every plausible pair the same: generate candidates, assign a score, then threshold. This plan takes a different route. It treats each predicted match as a **claim that must earn enough independent evidence**. The system routes records through several local evidence views, records both supporting and contradictory signals, and abstains by default when the evidence is thin. That fits macro $F_{0.5}$, where a false merge is more damaging than a missed link.

Core rule: do not use external business data, APIs, geocoding, or web lookups. Everything below uses only the provided TSVs.

What makes this approach distinct

Component	Typical approach	Our precision-first approach
Candidate generation	One broad fuzzy-search list	Multi-view local retrieval: name, address-number, rare-token, and character-shape blocks. Keep provenance for every candidate.
Decision	One similarity score	Evidence ledger: positive signals, negative signals, and a confidence gate. A high name score cannot silently override an address conflict.
Singletons	Implicitly treated as low score	Explicit abstention class: predict empty unless a candidate clears the precision gate.
Validation	Pair-level split	Hold out complete Source 1 entities; inspect candidate recall and macro F0.5 separately.

The ordered roadmap

Phase	Purpose	Required output before proceeding
0. Guardrails	Make the work reproducible and compliant	Rules checklist, fixed folders, version log
1. Understand	Profile data and form a validation protocol	EDA report + frozen Source 1 holdout
2. Retrieve	Build high-recall local candidate views	Candidate recall / reduction-ratio report
3. Decide	Score candidates using evidence and contradiction	Baseline macro F0.5 + error slices
4. Learn	Train a compact pair classifier	Thresholded validated model
5. Stress-test	Try only evidence-backed improvements	Chosen final version
6. Submit	Produce compliant output and archive	PASS validator + reproducible ZIP

Phase 0 - Guardrails and workspace

Do this once, before modelling.

Step	Action	Why it matters
0.1	Preserve the supplied student_resource folder. Create separate work/, output/, and experiment logs.	Keeps source material untouched and makes the final package reproducible.
0.2	Use only local data and local text normalization. No online enrichment of names, addresses, or countries.	Required by fair-play rules; violations are disqualifying.
0.3	Make every run take a seed and write its configuration, runtime, candidate count, F0.5, and output path.	Prevents accidental leaderboard chasing and preserves version history.
0.4	Reserve portal attempts: baseline only after validator PASS; then use the daily quota for materially different validated versions.	There are only five submissions per day.

Phase 1 - Learn the dataset before choosing a model

1. Read every TSV with a tab separator. Confirm Source 1/2/3 IDs, count countries, quantify missing names and addresses, measure duplicates, and calculate the fraction of Source 1 singletons and matches per entity.
2. Create a deterministic validation split on **Source 1 IDs**, for example 80% development / 20% validation. All labels for a held-out Source 1 entity stay held out. Never split individual pair rows because that leaks the entity's match pattern.
3. Implement exact per-entity $F_{0.5}$ and report it as a macro average. Also report precision, recall, singleton accuracy, candidate recall, and candidate reduction ratio. F0.5 is the selection metric; the others explain why it moves.

Decision gate

Do not build a complex model until the audit and metric agree with hand-checked examples from both US and India. France must remain an unseen open-set test country; never one-hot or hard-code country labels.

Phase 2 - Multi-view candidate routing

For each Source 1 record, create several compact candidate lists from Sources 2 and 3. Union them, deduplicate IDs, and record *which view retrieved each candidate*. This provenance later becomes a model feature and makes the candidate file auditable.

View	Local key	What it recovers	Risk control
Rare-name	Two rare normalized name tokens	Near-exact names despite punctuation/suffix variation	Ignore high-frequency tokens
Address anchor	Address number + rare street/locality token	Similar address with name variation	Require a second signal later
Character shape	Name/address character n-gram retrieval	Typos, abbreviations, transliteration-like changes	Top-k cap per view
Cross-field	Name token plus address token	High-precision straightforward pairs	Country may assist but is never required

Phase 3 - The evidence ledger decision layer

Each candidate pair receives a ledger rather than a single opaque score. The ledger has three buckets:

Evidence type	Examples	Use in final decision
Positive	High name similarity, rare token overlap, compatible address number, shared locality tokens, agreement across two candidate views	Raises confidence
Negative	Conflicting street number, incompatible country string when both known, low overlap in both name and address	Reduces or blocks confidence
Uncertainty	Missing fields, only one weak retrieval view, common business-name tokens	Raises the threshold or leads to abstention

Simple decision policy

Predict a match only when (a) the candidate is retrieved by a credible view, (b) the positive evidence exceeds the threshold, and (c) no hard contradiction exists. When evidence is ambiguous, output an empty match list. This is intentionally conservative because singletons score 1.0 when correctly left empty.

Phase 4 - Learn from labelled candidate pairs

1. Build candidate pairs on the development split. Label a pair positive only if its Source 2/3 ID occurs in the corresponding training ground-truth list.
2. Keep all positives and sample **hard negatives**: plausible candidates that resemble the Source 1 record but are not labelled matches. These teach the model the difference between a common name and an actual entity.
3. Train a compact, local, reproducible classifier on numeric evidence features. A gradient-boosted tree or calibrated logistic model is appropriate; no large external model is required.
4. Select the threshold by macro F0.5 on held-out Source 1 entities. Consider a stricter threshold for candidates with address conflicts. Do not tune against portal feedback.

Phase 5 - Controlled improvements

Order	Experiment	Keep it only if
5.1	Increase candidate recall with one additional local retrieval view	Candidate recall improves without harming runtime or precision after thresholding
5.2	Add one feature family: token rarity, numeric agreement, or retrieval provenance	Held-out macro F0.5 improves across at least two deterministic splits
5.3	Add contradiction penalties	False merges decrease and singleton performance stays stable
5.4	Blend two genuinely different models	Their validation errors are complementary and the blend improves F0.5

Stop rule: abandon a branch after two controlled variants fail to improve validation. Spend time on error slices, not broad random sweeps.

Final inference and submission

Step	Action	Proof
6.1 Freeze	Freeze features, retrieval views, threshold, seed, package versions, and chosen experiment ID.	Saved config and run log
6.2 Retrain	Fit the selected model on all provided training labels. Never incorporate portal score as a label.	Reproducible training command
6.3 Infer	Create final candidates for every test Source 1 entity; score candidates; output only accepted claims.	Both TSV outputs
6.4 Validate	Run the official validator. First normal validation; then --check-ids when memory permits.	PASS log
6.5 Archive	Copy runnable source, pinned requirements, outputs, filled methodology, and submission log into the prescribed ZIP structure.	Reproducible ZIP
6.6 Upload	Upload matching_results.tsv only after PASS. Record time, version, local F0.5, portal score, and checksum.	Submission log entry

Submission checklist

- Every test Source 1 ID appears exactly once in both output files.
- Files are UTF-8 TSV with the exact required headers.
- Every final match exists in the corresponding candidate list.
- No Source 1 IDs, unknown IDs, or duplicates appear in result lists.
- The output version has a recorded validation F0.5 and a completed validator PASS.
- The final ZIP contains code, pinned dependencies, outputs, and the completed methodology template.
- No external lookup, enrichment, or prohibited service was used anywhere in the pipeline.

A simple 72-hour cadence

Window	Focus	Decision
Hours 0-8	Audit, split, metric, first candidate baseline	No upload until validator PASS
Hours 8-28	Candidate recall and evidence-ledger features	First conservative submission only if validated
Hours 28-52	Pair classifier, thresholding, controlled error analysis	Submit only a meaningful improvement
Hours 52-68	Retrain, stress-test, final candidate audit	Choose robust version, not the noisiest public score
Hours 68-72	Final package, documentation, validator, version log	Final upload and archive

The goal is not to predict every possible link. The goal is to make each accepted link defensible, reproducible, and strong enough to survive private-leaderboard evaluation.